

Sprint 3: Tính năng Xác thực & Ghi chú riêng tư (Private Notes)

Thẻ 1: [3h] (BE) API Xác thực & CRUD Ghi chú riêng tư

 **Người phụ trách:** Backend Dev

 **Mô tả:**

Tạo API để kiểm tra mật khẩu do người dùng nhập vào có khớp với mật khẩu đã lưu ở Sprint 1 hay không. Đồng thời, sao chép logic CRUD của Note thường để áp dụng cho file dữ liệu vùng kín (data/private.json).

Checklist: Xác thực (Auth)

- ☐ **API POST** /api/private/auth: Nhận password từ Frontend gửi lên.
- ☐ Dùng fs.readFileSync đọc file profile.json để lấy mật khẩu gốc ra so sánh.
- ☐ Trả về JSON { "success": true } nếu khớp, hoặc { "success": false, "message": "Sai mật khẩu" } nếu không khớp.

Checklist: CRUD Note Kín (Copy logic Sprint 2)

- ☐ Tạo sẵn file data/private.json chứa mảng rỗng [].
- ☐ Tạo các API: **GET, POST, PUT, DELETE** cho đường dẫn /api/private/notes (Không cần tham số :topic như note thường vì note kín gom chung 1 file).
- ☐ Logic đọc/ghi y hệt như Sprint 2, chỉ thay đường dẫn trỏ tới file private.json.
- ☐ Dùng Postman test thử API Auth (nhập đúng/sai pass) và API thêm note kín.

Thẻ 2: [3h] (FE) UI Xác thực (Guard) chặn Vùng kín

 **Người phụ trách:** Frontend Dev

 **Mô tả:**

Xây dựng lớp bảo vệ (Guard) trên giao diện. Khi người dùng bấm vào tab "Vùng kín" (/private), họ không được thấy dữ liệu ngay mà phải qua màn hình nhập mật khẩu.

Checklist:

- ☐ Trong Component trang Private, tạo state quản lý trạng thái mở khóa: const [isUnlocked, setIsUnlocked] = useState(false).
- ☐ **UI Khóa:** Vẽ một Form nhỏ ra giữa màn hình gồm 1 Input (type="password") và 1 nút "Mở khóa". Form này chỉ hiện khi isUnlocked là false.

HỆ THỐNG QUẢN LÝ GHI CHÚ VÀ THÔNG TIN RIÊNG TƯ

- [] **Ghép API Auth:** Khi user bấm "Mở khóa", gọi API POST /api/private/auth.
- [] Nếu API trả về success: false -> Dùng alert() báo "Sai mật khẩu", xóa rỗng ô input.
- [] Nếu API trả về success: true -> Set setIsUnlocked(true) -> Form tự ẩn đi để nhường chỗ cho phần hiển thị Note kín.

Thẻ 3: [3h] (FE) UI Quản lý Ghi chú riêng tư (Tái sử dụng code)

 **Người phụ trách:** Frontend Dev

 **Mô tả:**

Hiển thị danh sách và Form thao tác cho các ghi chú kín. Hãy import lại Component danh sách Note của trang Ghi chú thường để xài lại, tránh viết lại HTML/CSS từ đầu.

Checklist:

- [] Bố trí phần render: Khi isUnlocked === true, gọi Component hiển thị danh sách Note ra màn hình.
- [] **Ghép API GET:** Gọi API GET /api/private/notes để lấy dữ liệu đổ ra giao diện.
- [] **Ghép API POST/PUT/DELETE:** Đổi các endpoint lúc Thêm/Sửa/Xóa thành /api/private/notes/.... (Lưu ý: Không cần truyền biến :topic như ở Sprint 2).
- [] Xử lý bảo mật FE: Đảm bảo khi F5 lại trang (/private), biến isUnlocked tự động bị reset về false (bắt người dùng nhập lại mật khẩu để đảm bảo an toàn).

Thẻ 4: [2h] (PM) Nghiệm thu tính năng (Testing Sprint 3)

 **Người phụ trách:** PM

 **Mô tả:**

Kiểm tra luồng bảo mật. PM cần cố gắng "đóng vai kẻ xấu" để xem ứng dụng có dễ dàng bị vượt qua lớp mật khẩu hay không.

Checklist:

- [] Mở ứng dụng, bấm vào tab Vùng kín. Giao diện có hiện form bắt nhập pass không? (Hay bị lỗi lộ dữ liệu ra trước 1-2 giây rồi mới che lại?).
- [] Cố tình nhập sai mật khẩu 3 lần xem thông báo lỗi có hiển thị đúng không.
- [] Nhập đúng mật khẩu -> Thêm mới 1 ghi chú riêng tư nhạy cảm.

HỆ THỐNG QUẢN LÝ GHI CHÚ VÀ THÔNG TIN RIÊNG TƯ

- [] Bấm phím F5 để tải lại trang -> Ứng dụng phải khóa lại như cũ và bắt nhập pass.
- [] Vào tab "Cài đặt", đổi một mật khẩu mới -> Quay lại tab Vùng kín, dùng mật khẩu cũ mở (phải báo lỗi), dùng mật khẩu mới mở (phải thành công).
- [] Nếu có lỗi bảo mật -> Comment tag @FE / @BE. Nếu trơn tru -> Kéo thẻ sang Done!
- [] **Trước khi Dev làm xong:** PM phải vào file Google Sheets viết sẵn các kịch bản Testcase (Khoảng 5-7 dòng cho Note thường, 5-7 dòng cho Note kín).
- [] **Khi Dev làm xong:** PM tiến hành thao tác trên web đúng theo các bước trong file Testcase.
- [] Cập nhật trạng thái Pass/Fail vào file Sheets.
- [] **Quy trình báo lỗi:** Nếu có thẻ nào Fail -> Bắt buộc phải tạo Card mới trên Trello (Cột Bugs), ghi rõ *Các bước tái hiện*, *Kết quả mong đợi*, *Kết quả thực tế* và tag tên Dev vào sửa. Không báo lỗi qua tin nhắn rác.

Hướng dẫn thực hiện Sprint 3: Tính năng Xác thực & Ghi chú riêng tư (Private Notes)

Đặc biệt, phần cuối sẽ mô phỏng một **tình huống xung đột mã nguồn (Merge Conflict)** kinh điển và cách giải quyết từng bước để nhóm bạn không bị hoảng loạn khi gặp phải.

PHẦN 1: THỰC HIỆN BACKEND (Cập nhật server.js)

Trong Sprint này, BE Dev sẽ thêm chức năng kiểm tra mật khẩu và thao tác với file private.json. Hãy copy khối mã dưới đây dán vào server.js (ngay trên dòng app.listen).

```
/**
 * =====
 * MODULE: BẢO MẬT & GHI CHÚ RIÊNG TƯ (PRIVATE NOTES)
 * Author: [Điền tên Backend Dev]
 * Date: [Ngày thực hiện]
 * Description: API kiểm tra mật khẩu và quản lý file private.json
 * =====
 */

const privateNotesFile = path.join(__dirname, 'data', 'private.json');
// Khởi tạo file private.json nếu chưa tồn tại
if (!fs.existsSync(privateNotesFile)) {
  fs.writeFileSync(privateNotesFile, '[]', 'utf8');
}

// 1. API Xác thực mật khẩu
app.post('/api/private/auth', (req, res) => {
  try {
    const profile = JSON.parse(fs.readFileSync(profilePath, 'utf8'));
    // Kiểm tra pass truyền lên có khớp với pass trong profile không
    if (profile.password === req.body.password) {
      res.json({ success: true });
    } else {
      res.status(401).json({ success: false, message: "Sai mật khẩu!" });
    }
  } catch (error) {
    res.status(500).json({ message: "Lỗi hệ thống xác thực" });
  }
});

// 2. API Lấy danh sách Ghi chú riêng tư
app.get('/api/private/notes', (req, res) => {
  try {
    const data = fs.readFileSync(privateNotesFile, 'utf8');
    res.json(JSON.parse(data));
  } catch (error) {
  }
});
```

HỆ THỐNG QUẢN LÝ GHI CHÚ VÀ THÔNG TIN RIÊNG TƯ

```
        res.status(500).json({ message: "Lỗi đọc ghi chú riêng tư" });
    }
});
// 3. API Thêm Ghi chú riêng tư
app.post('/api/private/notes', (req, res) => {
    try {
        let notes = JSON.parse(fs.readFileSync(privateNotesFile, 'utf8'));
        const newNote = {
            id: Date.now().toString(),
            title: req.body.title || "Lưu bút mật",
            content: req.body.content || "",
            createdAt: new Date().toISOString(),
            updatedAt: new Date().toISOString()
        };
        notes.push(newNote);
        fs.writeFileSync(privateNotesFile, JSON.stringify(notes, null, 2), 'utf8');
        res.json({ success: true, note: newNote });
    } catch (error) {
        res.status(500).json({ message: "Lỗi thêm ghi chú kín" });
    }
});
```

// (Tương tự, BE Dev tự viết thêm app.put và app.delete cho /api/private/notes/:id dựa trên logic Sprint 2 nhé)

PHẦN 2: THỰC HIỆN FRONTEND (ReactJS)

Tạo file mới frontend/src/PrivateNotes.jsx. File này chia làm 2 trạng thái: **Màn hình Khóa** (khi chưa nhập pass) và **Màn hình Danh sách** (sau khi nhập đúng).

```
/**
 * =====
 * COMPONENT: VÙNG KÍN & BẢO MẬT (PrivateNotes.jsx)
 * Author: [Điền tên Frontend Dev]
 *
 * [LƯU Ý]: Sử dụng lại phần lớn UI từ Notes.jsx. Thêm state isUnlocked để làm "cửa
bảo vệ".
 * =====
 */
import React, { useState, useEffect } from 'react';
function PrivateNotes() {
    /* =====
        VÙNG 1: STATE (Trạng thái)
        ===== */
    const [isUnlocked, setIsUnlocked] = useState(false); // Cờ khóa màn hình
```

HỆ THỐNG QUẢN LÝ GHI CHÚ VÀ THÔNG TIN RIÊNG TƯ

```
const [passwordInput, setPasswordInput] = useState('');

const [notes, setNotes] = useState([]);
const [formData, setFormData] = useState({ id: null, title: '', content: '' });
/* =====
    VÙNG 2: LOGIC (Xác thực & Fetch Data)
    ===== */
// Kiểm tra mật khẩu
const handleLogin = () => {
    fetch('http://localhost:5000/api/private/auth', {
        method: 'POST',
        headers: { 'Content-Type': 'application/json' },
        body: JSON.stringify({ password: passwordInput })
    })
    .then(res => res.json())
    .then(data => {
        if (data.success) {
            setIsUnlocked(true); // Mở khóa
            fetchPrivateNotes(); // Lấy dữ liệu
        } else {
            alert("Sai mật khẩu, vui lòng thử lại!");
            setPasswordInput('');
        }
    });
};

const fetchPrivateNotes = () => {
    fetch('http://localhost:5000/api/private/notes')
    .then(res => res.json())
    .then(data => setNotes(data));
};

const handleSave = () => {
    fetch('http://localhost:5000/api/private/notes', {
        method: 'POST',
        headers: { 'Content-Type': 'application/json' },
        body: JSON.stringify({ title: formData.title, content: formData.content })
    }).then(() => {
        fetchPrivateNotes();
        setFormData({ id: null, title: '', content: '' });
    });
};
/* =====
    VÙNG 3: RENDER (Hiển thị)
    ===== */
// 3.1. Nếu chưa mở khóa -> Render màn hình nhập Pass
```

HỆ THỐNG QUẢN LÝ GHI CHÚ VÀ THÔNG TIN RIÊNG TƯ

```
if (!isUnlocked) {
  return (
    <div style={{ padding: '50px', textAlign: 'center' }}>
      <h2>Khu vực Bảo mật</h2>
      <p>Vui lòng nhập mật khẩu để truy cập</p>
      <input
        type="password"
        value={passwordInput}
        onChange={(e) => setPasswordInput(e.target.value)}
        placeholder="Nhập mật khẩu..."
      />
      <button onClick={handleLogin} style={{ marginLeft: '10px' }}>Mở khóa</button>
    </div>
  );
}
// 3.2. Nếu đã mở khóa -> Render giao diện Note tương tự Sprint 2
return (
  <div style={{ padding: '20px', backgroundColor: '#ffebee' }}>
    <h2 style={{ color: 'red' }}>Khu vực Ghi chú Riêng tư 🔒</h2>

    {/* Form nhập liệu */}
    <div style={{ border: '1px solid red', padding: '10px', marginBottom: '20px'
  >>>
      <input
        placeholder="Tiêu đề bí mật" value={formData.title}
        onChange={e => setFormData({...formData, title: e.target.value})}
        style={{ display: 'block', width: '100%', marginBottom: '10px' }}
      />
      <textarea
        placeholder="Nội dung bí mật" value={formData.content}
        onChange={e => setFormData({...formData, content: e.target.value})}
        style={{ display: 'block', width: '100%', height: '80px', marginBottom:
'10px' }}
      />
      <button onClick={handleSave} style={{ backgroundColor: 'red', color: 'white'
  >>>Lưu bí mật</button>
    </div>
    {/* Danh sách */}
    <div style={{ display: 'grid', gridTemplateColumns: '1fr 1fr', gap: '15px' }}>
      {notes.map(note => (
        <div key={note.id} style={{ border: '1px solid red', padding: '15px' }}>
          <h4>{note.title}</h4>
          <p>{note.content}</p>
        </div>
      ))}
    </div>
  </div>
);
```

```
    )}}  
  </div>  
</div>  
  );  
}  
export default PrivateNotes;
```

PHẦN 3: TÌNH HUỐNG XUNG ĐỘT MÃ NGUỒN (MERGE CONFLICT)

Xung đột (Merge Conflict) xảy ra khi 2 lập trình viên cùng sửa **chung một dòng code** trên **cùng một file**, và Git không biết phải ưu tiên lấy code của ai.

Kịch bản xảy ra lỗi:

1. **Bạn A (Backend)** thêm dòng code in ra chữ "API Private đã chạy!" ở cuối file server.js và git push thành công lên Github.
2. Cùng lúc đó, **Bạn B (Frontend)** cũng đang mớ mớ, mở file server.js ra sửa lại cổng chạy từ 5000 thành 5001 ở ngay cùng dòng cuối cùng đó.
3. Bạn B code xong, làm theo đúng quy trình:

```
git add .  
git commit -m "Đổi port sang 5001"  
git pull origin main
```

4. **BÙM!** Git báo lỗi màu đỏ lừ:

CONFLICT (content): Merge conflict in backend/server.js

Automatic merge failed; fix conflicts and then commit the result.

Cách Giải Quyết Xung Đột (Từng bước một):

Bước 1: Giữ bình tĩnh và không hoảng loạn

Đừng vội gõ các lệnh linh tinh. Lỗi này rất bình thường. Git đã tạm dừng quá trình gộp code và đánh dấu điểm bị lỗi cho bạn.

Bước 2: Mở file bị lỗi (server.js) lên

Khi mở bằng VS Code, bạn sẽ thấy một đoạn mã kỳ lạ xuất hiện với các dấu <<<<<<<, =====, >>>>>>:

```
<<<<<< HEAD  
app.listen(5001, () => console.log("Backend chạy port 5001"));  
=====  
app.listen(5000, () => {  
  console.log("Backend chạy port 5000");
```

HỆ THỐNG QUẢN LÝ GHI CHÚ VÀ THÔNG TIN RIÊNG TƯ

```
console.log("API Private đã chạy!");  
});  
>>>>>> a1b2c3d4e5f6... (ID commit của Bạn A)
```

- **Phần từ <<<<<< HEAD đến =====:** Là code hiện tại trên máy của BẠN B.
- **Phần từ ===== đến >>>>>>:** Là code mới tải từ GITHUB về (của BẠN A).

Bước 3: Lựa chọn (Sửa file bằng tay)

Nhiệm vụ của bạn là xóa đi các dấu git tạo ra và giữ lại đoạn code đúng nhất (hoặc gộp cả 2 lại). Ở đây, ta quyết định giữ port 5000 của A và xóa thay đổi của B. Bạn sửa lại file sạch sẽ như sau:

```
app.listen(5000, () => {  
  console.log("Backend chạy port 5000");  
  console.log("API Private đã chạy!");  
});
```

(Lưu file lại).

Bước 4: Xác nhận đã sửa xong

Mở lại Terminal, báo cho Git biết bạn đã giải quyết xong xung đột bằng cách add và commit lại:

```
git add backend/server.js  
git commit -m "fix: Giai quyet conflict o file server.js"
```

Bước 5: Đẩy code lên

Bây giờ xung đột đã tan biến, bạn có thể đẩy code lên an toàn:

```
git push origin main
```

Bài học rút ra: Luôn git pull trước khi bắt đầu code một tính năng mới. Nếu chia vùng comment rõ ràng như ở Sprint 2, việc cả 2 người cùng sửa chung 1 dòng sẽ rất hiếm khi xảy ra!